

Artificial Intelligence and Data Science

CSA1008 – Software Engineering

Assignment Title: CO3-AT1-Git & Docker Technical Assessment

Case Study Title: Smart Waste Collection System

(SDG 11: Sustainable Cities and Communities)

Name: K. Dedeepya

Reg No: 192324252

Class & Section: 4th year B. Tech AIDS

Faculty Name: Sharmili K

Date of Submission: 15-08-26

1. Problem Analysis

1.1 Problem Statement

Municipal authorities generally collect waste using fixed schedules without considering the actual fill level of garbage bins. This can result in two major problems. Nearly full or overflowing bins may remain uncollected and cause hygiene problems, while partially empty bins may be collected unnecessarily, resulting in additional fuel consumption and operational cost.

The proposed Smart Waste Collection System provides a software-based solution for managing waste collection activities. The system organizes waste-bin information, collection-related data, and application services in a structured manner. The system can be extended to monitor bin status and support optimized collection planning. Git is used to manage the source code and maintain different versions of the project. Docker is used to package and run the application components in isolated containers.

1.2 Objectives

The main objectives are:

- To develop a software system for smart waste collection management.
- To maintain waste-bin information.
- To support efficient collection activities.
- To provide a structured frontend and backend.
- To maintain the project using Git.
- To use branching for collaborative development.

1.3 Functional Requirements

Requirement	Description
Waste Bin Management	Store basic information about waste bins
Location Management	Maintain the location of each bin
Fill-Level Management	Store the fill level of bins
Status Management	Maintain the current bin status
Collection Management	Support collection-related operations
Frontend Interface	Provide a simple user interface
Backend Service	Handle application requests
Database	Store structured waste-bin information
Containerization	Run application services using Docker

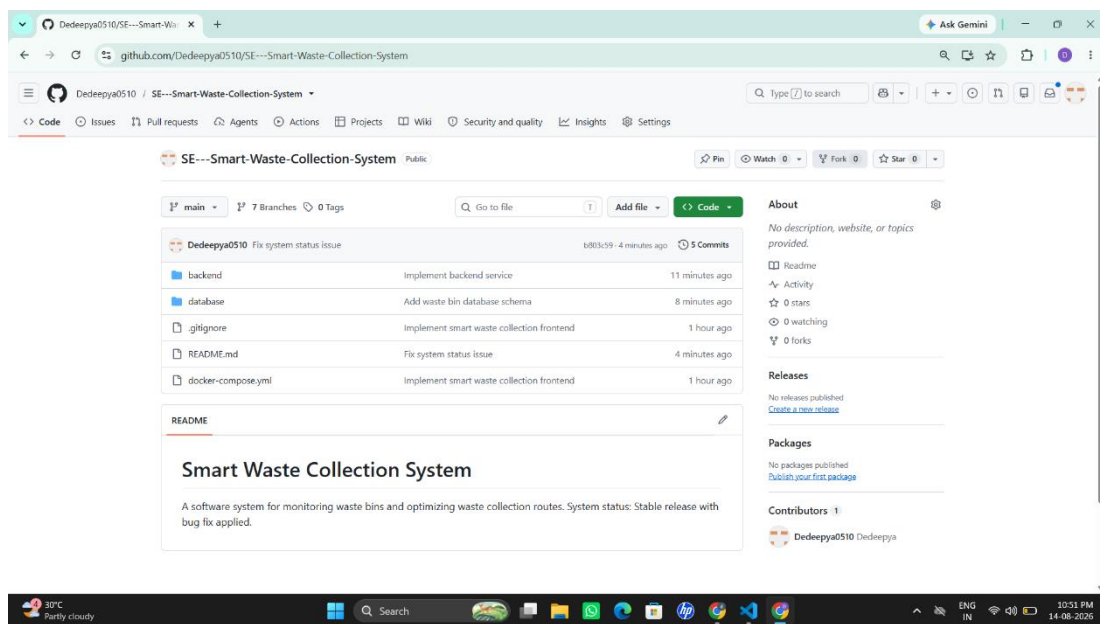
1.4 Non-Functional Requirements

- **Performance:** The system should respond quickly to user requests.
- **Scalability:** Additional waste bins and services should be supportable.
- **Maintainability:** Source code should be organized into separate components.
- **Portability:** The application should run consistently using Docker.
- **Reliability:** Application services should be restarted able independently.
- **Security:** Sensitive configuration should not be hard-coded.
- **Usability:** The interface should be simple and easy to use.

1.5 Expected Outcome

The expected outcome is a containerized Smart Waste Collection System with a frontend, backend service, and database structure. Git provides version control and collaboration, while Docker provides a consistent execution environment.

Screenshot 1: GitHub repository showing the Smart-Waste-Collection-System repository.



2. Project Structure Design

The project repository is organized as follows:

Smart-Waste-Collection-System/

```
|
|
| — frontend/
|   | — index.html
| — backend/
|   | — app.txt
```

```

├── database/
│   └── schema.sql
├── Dockerfile
├── docker-compose.yml
├── README.md
└── .gitignore

```

2.1 Folder and File Description

File/Folder	Purpose
frontend/	Contains frontend files
index.html	Provides the web interface
backend/	Contains backend-related files
app.txt	Contains backend service information used in the project
database/	Contains database-related files
schema.sql	Defines the waste-bin table
Dockerfile	Defines the frontend Docker image
docker-compose.yml	Defines application services and networking
README.md	Provides project information
.gitignore	Specifies files that should not be tracked by Git

2.2 Repository Organization

The repository follows a separation-of-concerns approach. Frontend, backend, and database-related resources are maintained in separate directories. Docker configuration is maintained at the root level so that the complete application can be started from one location.

2.3 Database Structure

The project uses a waste_bins table with the following fields:

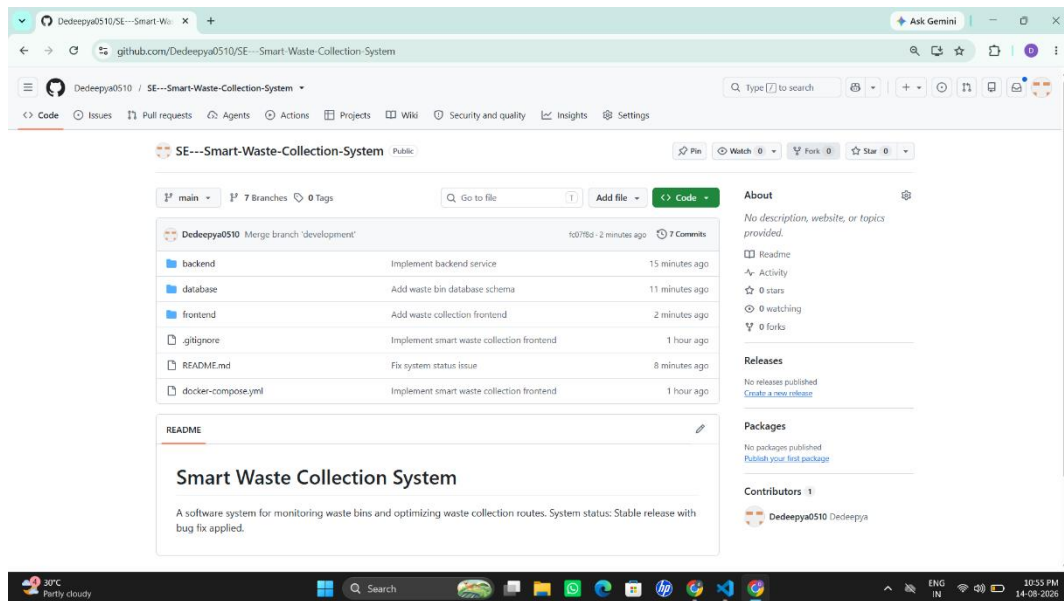
```

CREATE TABLE waste_bins (
    id INT PRIMARY KEY,
    location VARCHAR(100),
    fill_level INT,
    status VARCHAR(30)
);

```

The table stores the identifier, location, fill level, and current status of a waste bin.

Screenshot 2:



3. Git Repository Initialization

Git is used as the version control system for the project.

3.1 Repository Creation

The project folder was opened in VS Code and Git was initialized using:

- `git init`: The repository status was checked using:
- `git status`: The project files were added using:
- `git add.`: The initial version was committed using:
- `git commit -m "Initial project setup"`

3.2 Purpose of Git

Git maintains the complete history of source-code changes. It allows developers to create branches, commit changes, merge features, and synchronize the repository with GitHub.

3.3 Important Git Components

- **.git Directory**: The .git directory stores Git repository information such as commits, branches, and internal configuration.
- **.gitignore**: The .gitignore file prevents unnecessary files from being added to the repository.
- **README.md**: The README.md file contains project information, usage instructions, and documentation.

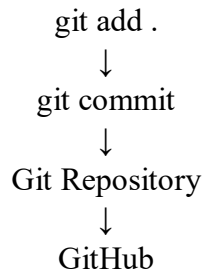
3.4 Git Repository Workflow

Project Files

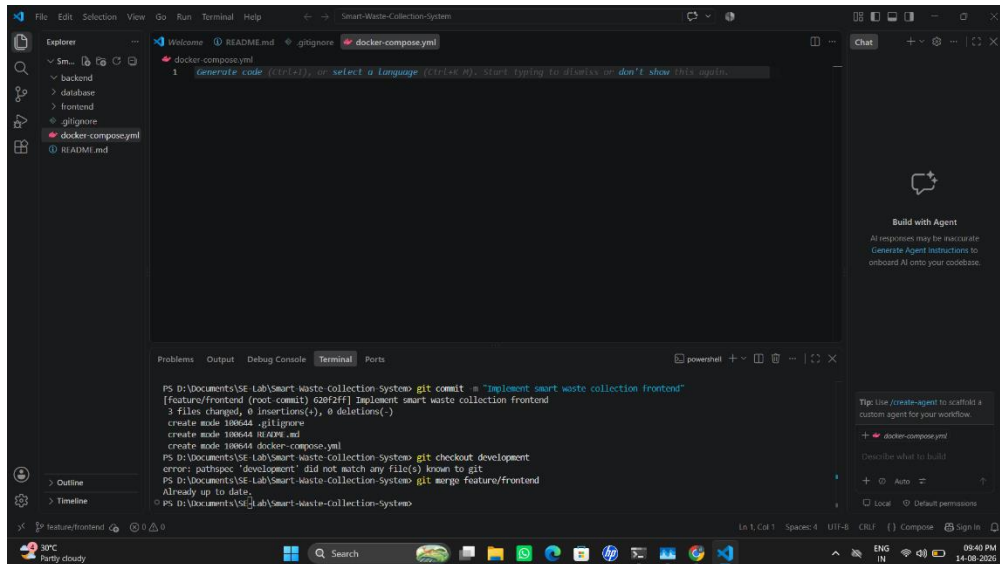


`git init`





Screenshot 3: Terminal showing git init and git status.



4. Git Branching Strategy

A structured branching strategy was selected for the Smart Waste Collection System.

4.1 Branches Used

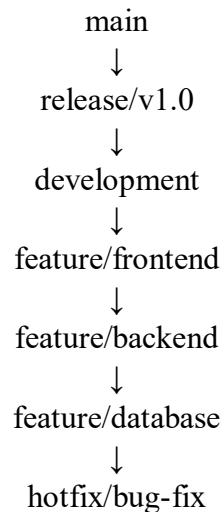
1. main
2. development
3. feature/frontend
4. feature/backend
5. feature/database
6. release/v1.0
7. hotfix/bug-fix

4.2 Branch Purpose

Branch	Purpose
main	Stable project version
development	Integrated development code
feature/frontend	Frontend development
feature/backend	Backend development

feature/database	Database development
release/v1.0	Release preparation
hotfix/bug-fix	Critical bug correction

4.3 Branching Structure



4.4 Justification

The selected branching structure separates stable code from development work. Features can be developed independently and merged into the development branch. Release preparation and urgent fixes are handled separately. This reduces the possibility of directly modifying the stable main branch during normal development.

Screenshot 4: Terminal showing: git branch with the project branches.

The screenshot shows a Visual Studio Code editor window with a terminal at the bottom. The terminal displays the output of the `git branch` command, listing the following branches: `main`, `release/v1.0`, `development`, `feature/frontend`, `feature/backend`, `feature/database`, and `hotfix/bug-fix`. The `main` branch is currently checked out. The editor's Explorer sidebar on the left shows the project structure, including folders for `backend`, `database`, `frontend`, and files like `.gitignore`, `docker-compose.yml`, and `README.md`. The central editor pane shows a SQL file named `schema.sql` with a `CREATE TABLE` statement for `waste_bins`.

5. Version Control Workflow

The project demonstrated practical Git operations including branch creation, commits, merging, and repository synchronization.

5.1 Feature Branch Creation

- Frontend development: `git switch -c feature/frontend`
- Backend development: `git switch -c feature/backend`
- Database development: `git switch -c feature/database`
- Release branch: `git switch -c release/v1.0`
- Hotfix branch: `git switch -c hotfix/bug-fix`

5.2 Commit Operations

- Frontend: `git commit -m "Add smart waste collection project description"`
- Backend: `git commit -m "Implement backend service"`
- Database: `git commit -m "Add waste bin database schema"`
- Hotfix: `git add README.md`
`git commit -m "Fix system status issue"`

5.3 Merge Operations

- Feature branches were merged into development:

`git switch development`

`git merge feature/frontend`

`git merge feature/backend`

`git merge feature/database`

- The release branch was merged into main:

`git switch main`

`git merge release/v1.0`

- The hotfix was also merged into main:

`git merge hotfix/bug-fix`

5.4 Repository Synchronization

The project was synchronized with GitHub using:

- `git push -u origin main`
- `git push -u origin development`
- `git push origin feature/frontend`
- `git push origin feature/backend`
- `git push origin feature/database`
- `git push origin release/v1.0`
- `git push origin hotfix/bug-fix`

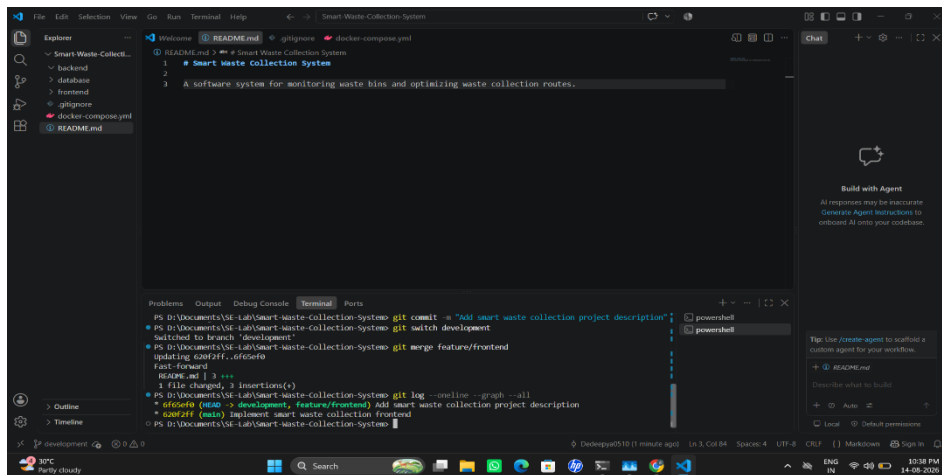
5.5 Git Workflow Diagram

Developer





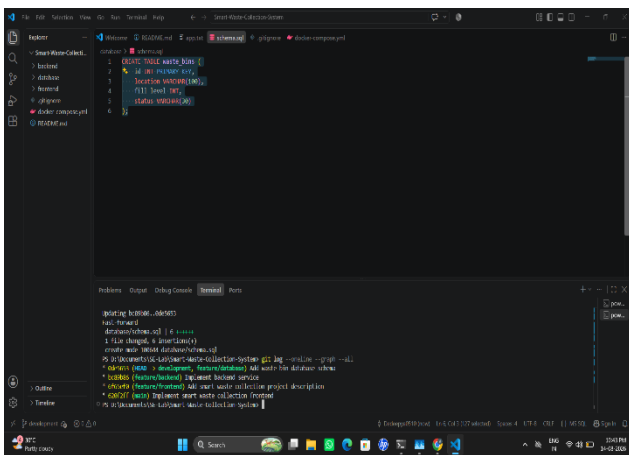
Screenshot 5: Terminal showing a successful merge.



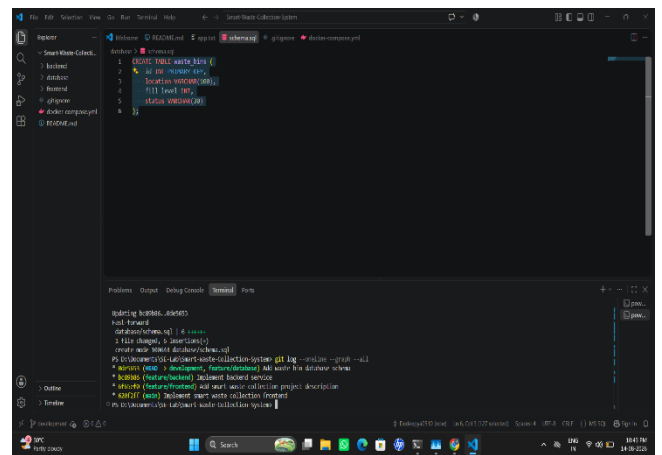
```
PS D:\Documents\VS-Lab\Smart-Waste-Collection-System> git commit -m "Add smart waste collection project description"
[main 6f65ef0] Add smart waste collection project description
1 file changed, 3 insertions(+)
```

```
PS D:\Documents\VS-Lab\Smart-Waste-Collection-System> git switch development
Switched to branch 'development'
PS D:\Documents\VS-Lab\Smart-Waste-Collection-System> git merge feature/frontend
Fast-forward
   6f65ef0 (HEAD -> development, feature/frontend) Add smart waste collection project description
   6f65ef0 (main) Implement smart waste collection frontend
1 file changed, 3 insertions(+)
```

```
PS D:\Documents\VS-Lab\Smart-Waste-Collection-System> git log --online --graph --all
* 6f65ef0 (HEAD -> development, feature/frontend) Add smart waste collection project description
* 6f65ef0 (main) Implement smart waste collection frontend
PS D:\Documents\VS-Lab\Smart-Waste-Collection-System>
```

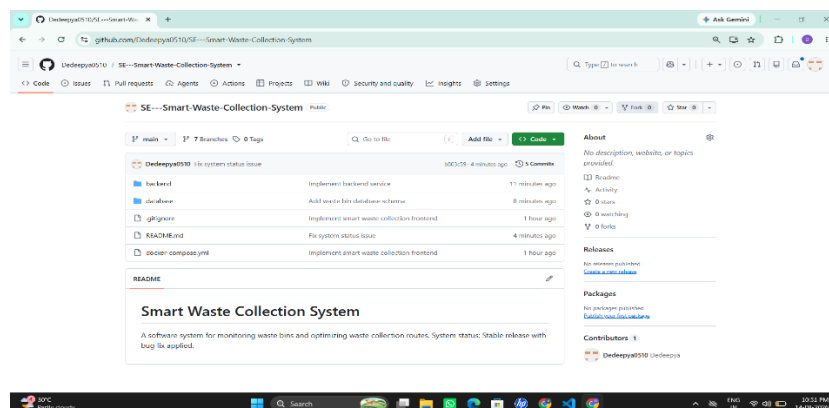


```
PS D:\Documents\VS-Lab\Smart-Waste-Collection-System> git merge feature/frontend
Fast-forward
   6f65ef0 (HEAD -> development, feature/frontend) Add smart waste collection project description
   6f65ef0 (main) Implement smart waste collection frontend
1 file changed, 3 insertions(+)
```



```
PS D:\Documents\VS-Lab\Smart-Waste-Collection-System> git log --online --graph --all
* 6f65ef0 (HEAD -> development, feature/frontend) Add smart waste collection project description
* 6f65ef0 (main) Implement smart waste collection frontend
PS D:\Documents\VS-Lab\Smart-Waste-Collection-System>
```

Screenshot 6: GitHub repository showing pushed branches and files.



6. Commit History Analysis

The project follows meaningful commit messages so that each change can be understood easily.

6.1 Commit History

The project contains commits such as:

- Initial project setup
- Add smart waste collection project description
- Implement backend service
- Add waste bin database schema
- Fix system status issue
- Add waste collection frontend
- Add Docker containerization
- The commit history was examined using: `git log --oneline`
- A graphical representation was generated using: `git log --oneline --graph --all`

6.2 Good Commit Message Practices

The commit messages describe the actual purpose of the change. Examples include:

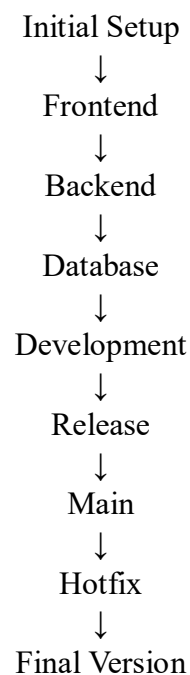
Implement backend service

Add waste bin database schema

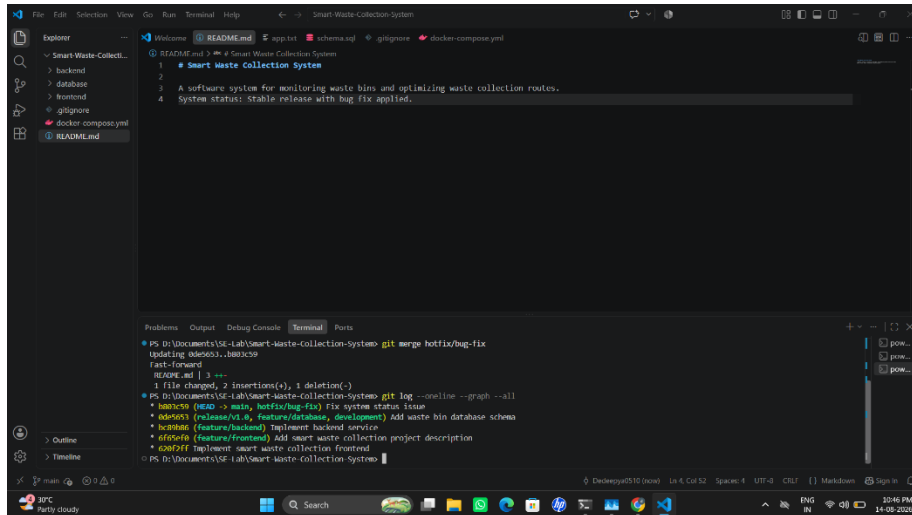
Fix system status issue

Add Docker containerization

6.3 Git History Diagram



Screenshot 7: `git log --oneline --graph --all`



7. Docker Environment Design

Docker is used to package and run application components in isolated containers.

7.1 Why Docker is Suitable

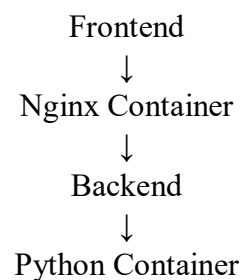
Docker provides a consistent environment for running the application. It packages the application together with the required runtime environment, reducing dependency-related problems.

For this project, Docker provides:

- Portability
- Environment consistency
- Service isolation
- Easier deployment
- Reproducible execution
- Easy service management

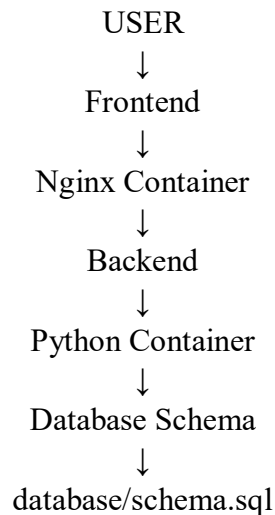
7.2 Components

The current containerized setup contains:



The database schema is maintained in the repository and can be extended into a dedicated database container in future development.

7.3 Docker Architecture



8. Dockerfile Development

The frontend was containerized using a Dockerfile.

8.1 Dockerfile

```
FROM nginx:alpine
```

```
COPY frontend/index.html /usr/share/nginx/html/index.html
```

```
EXPOSE 80
```

8.2 Instruction Explanation

- **FROM**

```
FROM nginx:alpine
```

Uses the lightweight Nginx Alpine image as the base image.

- **COPY**

```
COPY frontend/index.html /usr/share/nginx/html/index.html
```

Copies the project frontend file into the Nginx web-server directory inside the container.

- **EXPOSE**

```
EXPOSE 80
```

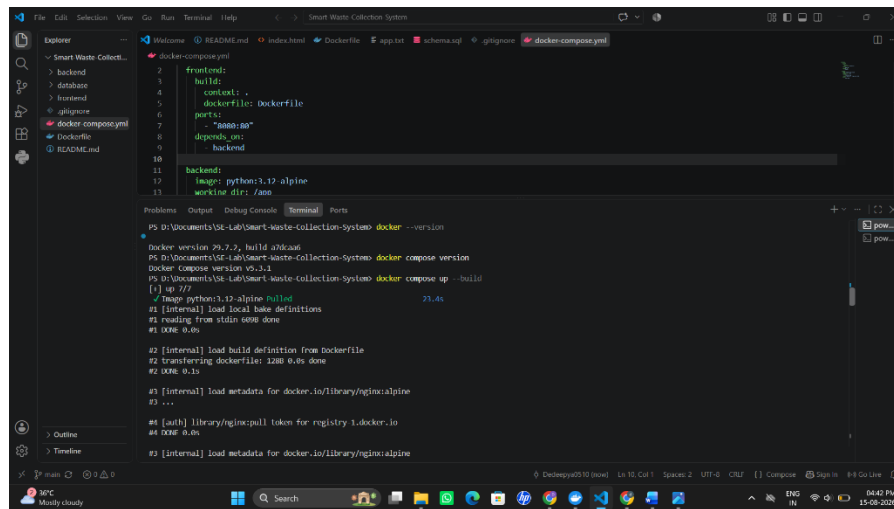
Indicates that the Nginx service listens on port 80 inside the container.

8.3 Docker Image Creation

The image was created through Docker Compose using: `docker compose up --build`

The Docker build successfully downloaded the Nginx image, copied the frontend file, and created the frontend image.

Screenshot 8: docker compose up --build output showing successful image build.



9. Docker Compose Design

Docker Compose is used to define and manage multiple services using one configuration file.

9.1 Docker Compose Configuration

services:

frontend:

build:

context: .

dockerfile: Dockerfile

ports: - "8080:80"

depends_on: - backend

backend:

image: python:3.12-alpine

working_dir: /app

volumes: - ./backend:/app

command: python -m http.server 5000

ports: - "5000:5000"

9.2 Services

Frontend Service

The frontend service is built from the project's Dockerfile.

frontend:

build:

context: .

dockerfile: Dockerfile

It maps:

Host Port 8080 → Container Port 80

Therefore, the frontend can be accessed using:

`http://localhost:8080`

Backend Service

The backend service uses Python 3.12 Alpine.

backend: image: python:3.12-alpine

It runs a Python HTTP server on port 5000.

Volumes

volumes: - ./backend:/app

This mounts the local backend directory into the container.

Dependencies

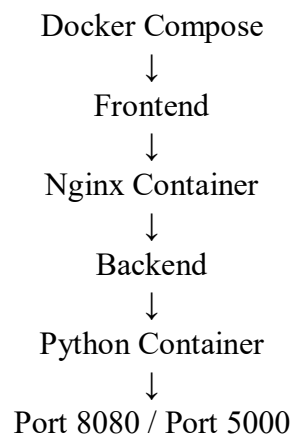
depends_on: - backend

This establishes the startup dependency between the frontend and backend services.

9.3 Networking

Docker Compose automatically creates a project network. The containers can communicate through this Docker-managed network.

9.4 Docker Compose Architecture



10. Container Deployment and Testing

10.1 Build and Start

The application was started using:

`docker compose up --build`

Docker performed the following operations:

1. Pulled the Python image.
2. Pulled the Nginx Alpine image.
3. Built the frontend image.
4. Created the Docker network.
5. Created the frontend container.
6. Created the backend container.
7. Started both services.

10.2 Verify Running Containers

The containers were checked using: `docker ps`

Expected services:

`smart-waste-collection-system-frontend-1`

`smart-waste-collection-system-backend-1`

10.3 Frontend Testing

The frontend was tested through: `http://localhost:8080`

The page displayed: Smart Waste Collection System

Waste monitoring and collection management system.

10.4 Backend Testing

The backend service runs on: `http://localhost:5000`

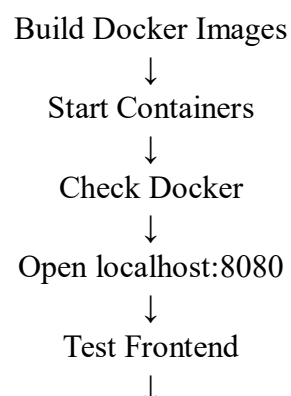
The Python HTTP service verifies that the backend container is running successfully.

10.5 Container Shutdown

After testing, the services were stopped using: `docker compose down`

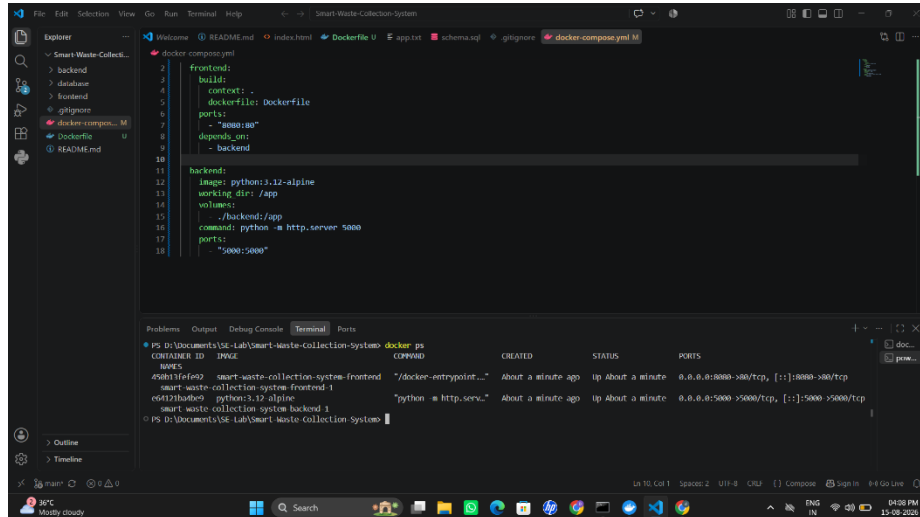
This removed the running containers and the Compose-created network.

10.6 Testing Flow

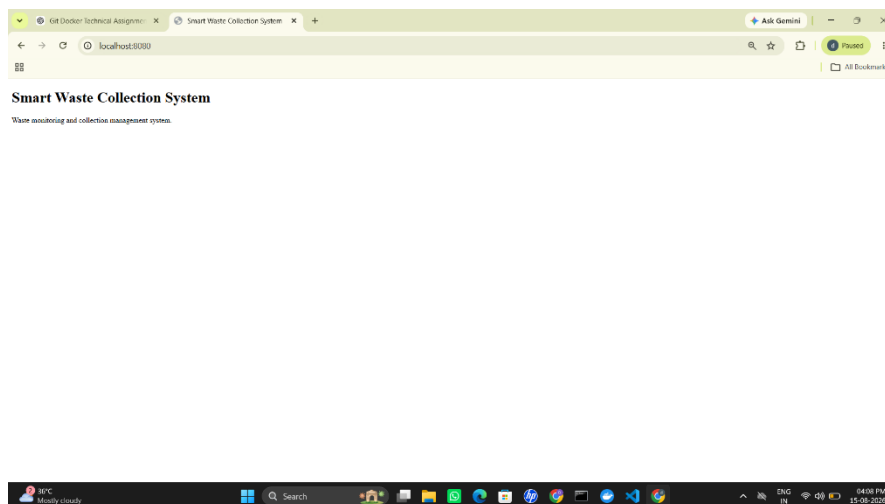


Stop Containers

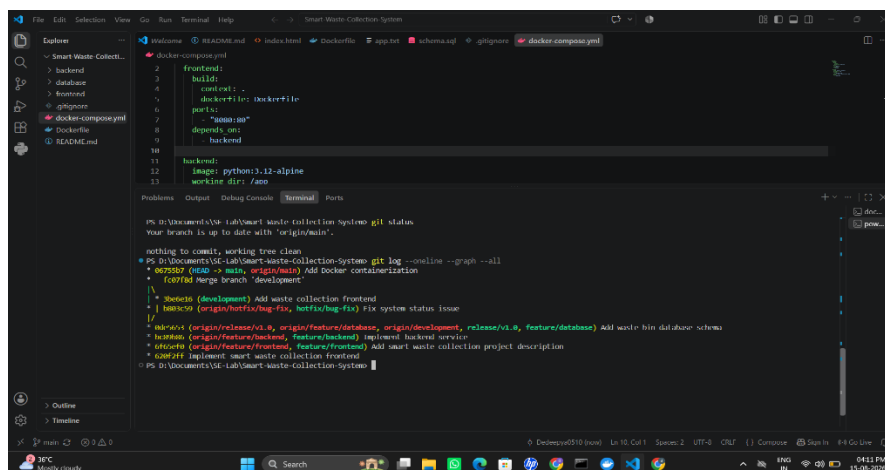
Screenshot 10: docker ps showing the running containers.



Insert Screenshot 11 here: Browser showing <http://localhost:8080>.



Insert Screenshot 12 here: docker compose down output.



11. Benefits of Git and Docker

11.1 Git Benefits

Collaboration: Different developers can work on different branches without directly modifying the stable code.

Version Management: Git records every committed change and allows previous versions to be examined.

Branching: Feature development can be separated from stable project code.

Rollback: Previous commits can be used to restore an earlier version if necessary.

Repository Synchronization: GitHub provides a central remote repository for sharing the project.

11.2 Docker Benefits

Portability: The application can run in a consistent containerized environment.

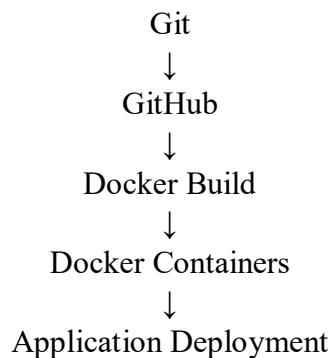
Isolation: Frontend and backend services can execute independently.

Deployment: Docker Compose simplifies starting the complete application.

Maintainability: Individual services can be modified without restructuring the entire project.

Scalability: Additional containers or services can be added as the system grows.

11.3 Combined Git and Docker Workflow



12. Challenges and Improvements

12.1 Challenges Encountered

Git Branch Management: Managing multiple branches requires careful switching and merging.

Merge Operations: Feature branches need to be merged into the correct target branch.

Docker Installation: Docker Desktop initially required additional system resources and the Docker Linux engine had to be started before containers could run.

Docker Engine: The Docker CLI was available before the Docker engine became operational. Once Docker Desktop and the Linux engine were running, Compose successfully built and started the services.

Container Configuration: The frontend and backend required separate service configurations and port mappings.

12.2 Improvements

Real Database Container: A MySQL or PostgreSQL container can be added to Docker Compose.

Real Backend API: The current backend can be replaced with a proper Flask, FastAPI, Node.js, or Spring Boot REST API.

Smart Bin Sensors: IoT sensors can provide real-time fill-level information.

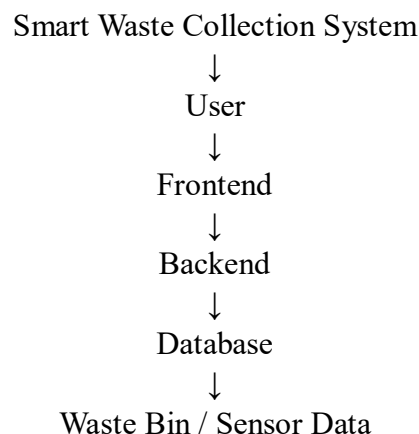
Route Optimization: Algorithms such as shortest path or vehicle-routing optimization can be integrated.

Authentication: Role-based authentication can be provided for administrators and collection staff.

Monitoring: Application health checks and centralized logging can be added.

CI/CD: GitHub Actions can automatically test, build, and deploy the Docker application after code changes.

13. System Architecture Diagram



Main Components

User: Municipal staff interact with the application.

Frontend: Displays waste collection information.

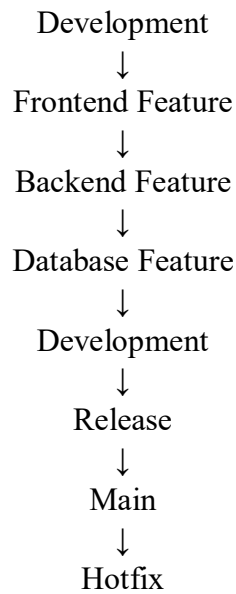
Backend: Handles application processing and requests.

Database: Stores waste-bin information.

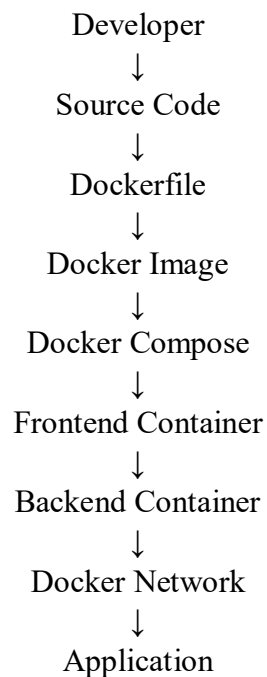
Waste Bin/Sensor Layer: Represents the future real-time source of bin fill-level information.

14. Git Workflow Diagram





15. Docker Workflow Diagram



18. Conclusion

The Smart Waste Collection System provides a software engineering solution to the problems caused by fixed waste collection schedules. The project is organized into frontend, backend, and database components.

Git was used to provide repository management, branching, commits, merging, version history, and GitHub synchronization. A branching strategy consisting of main, development, feature, release, and hotfix branches was implemented.

Docker was used to containerize the application. Docker file and Docker Compose were used to build and execute the application services. The frontend container successfully served the

Smart Waste Collection System interface, while the backend container provided the supporting backend service.

The combination of Git and Docker provides a structured approach to collaborative development, version control, portability, deployment, and maintainability. Future versions of the system can integrate real-time IoT sensors, a production database, route optimization algorithms, authentication, monitoring, and CI/CD automation.